

Schema in Mongoose vs Native MongoDB Driver

1. Mongoose

In Mongoose, a **schema** is **required**. Mongoose is an Object Data Modeling (ODM) library, meaning it works with a schema-based structure to manage the data and interaction with MongoDB.

- **Schemas** are used to define the structure of documents in a collection. They provide validation, type enforcement, default values, and instance methods. This makes your code more structured and predictable.
- **Advantages of using a schema in Mongoose:**
 - **Validation:** You can validate the data before saving it.
 - **Type Enforcement:** Schema enforces the type of data, ensuring consistency in your documents.
 - **Default Values:** You can define default values for fields.
 - **Custom Methods:** You can add methods to your models to interact with documents in a more readable and maintainable way.

Example:

js

CopyEdit

```
const mongoose = require('mongoose');

// Define a schema
const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true },
  age: { type: Number, default: 18 }
});

// Create a model
const User = mongoose.model('User', userSchema);

// Example: Creating a new user
const newUser = new User({
  name: 'John Doe',
  email: 'johndoe@example.com'
```

```
});
```

```
newUser.save()  
  .then(() => console.log('User saved!'))  
  .catch((err) => console.log(err));
```

Here, the **schema** defines the structure and validation of the User document.

2. Native MongoDB Driver

In the **Native MongoDB Driver**, you don't need to define a schema. The driver is more flexible and lower-level, so it does not enforce any structure or validation for your documents. You have complete freedom to insert, update, and query documents without having to define any schema upfront.

- **No Schema Enforcement:** MongoDB documents can have different structures within the same collection. This gives you flexibility but also places the responsibility on you to ensure consistency across documents.
- **Advantages of not using a schema:**
 - **Flexibility:** You can insert documents of any shape or structure without needing to define a schema.
 - **Faster for simple use cases:** If you don't need complex relationships or validation, the native driver can be quicker and more flexible.

Example:

```
js
```

CopyEdit

```
const { MongoClient } = require('mongodb');  
  
// MongoDB URI  
const uri = 'mongodb://localhost:27017';  
const dbName = 'myDatabase';  
  
async function connectToMongoDB() {  
  const client = new MongoClient(uri, { useNewUrlParser: true, useUnifiedTopology: true });  
  
  try {  
    await client.connect();  
    console.log('MongoDB connected');
```

```
const db = client.db(dbName);

const usersCollection = db.collection('users');


// Insert a user document without any predefined schema
const userData = { name: 'John Doe', email: 'johndoe@example.com', age: 30 };
const result = await usersCollection.insertOne(userData);
console.log('User created:', result.insertedId);


} catch (err) {
  console.log('Error:', err);
}
}


// Call the function
connectToMongoDB();
```

In the native MongoDB driver example, there's no schema. You directly interact with the database and its collections. The driver will allow any kind of data structure to be stored in the collection.

Summary

- **Mongoose** requires a **schema** to define the structure and validation of documents in a collection. It's best suited for applications where you need data consistency, structure, and validation.
- **Native MongoDB Driver** does **not require a schema** and provides greater flexibility. You can insert any document shape, but you lose the features that come with a schema, such as validation and data consistency.

In short, **schema is not necessary** in the native MongoDB driver, but **it is needed in Mongoose** to make the most of its features.

Steps to Connect to Multiple Databases Using Mongoose:

1. Install Mongoose (if you haven't already):

bash

CopyEdit

```
npm install mongoose
```

2. Example Code:

Here, I'll show you how to connect to two databases, acs and dcs, using Mongoose in a Node.js application. You'll establish separate connections for each database.

js

CopyEdit

```
const mongoose = require('mongoose');
```

```
// MongoDB URIs for the two databases
```

```
const acsUri = 'mongodb://localhost:27017/acs'; // Connection string for ACS database
```

```
const dcsUri = 'mongodb://localhost:27017/dcs'; // Connection string for DCS database
```

```
// Connecting to the 'acs' database
```

```
mongoose.createConnection(acsUri, { useNewUrlParser: true, useUnifiedTopology: true })
```

```
.then(() => {
```

```
  console.log('Connected to ACS database');
```

```
})
```

```
.catch((err) => {
```

```
  console.log('Error connecting to ACS database:', err);
```

```
});
```

```
// Connecting to the 'dcs' database
```

```
mongoose.createConnection(dcsUri, { useNewUrlParser: true, useUnifiedTopology: true })
```

```
.then(() => {
```

```
  console.log('Connected to DCS database');
```

```
})
```

```
.catch((err) => {  
  console.log('Error connecting to DCS database:', err);  
});
```

```
// Example Schema and Model for the 'acs' database
```

```
const acsConnection = mongoose.createConnection(acsUri, { useNewUrlParser: true,  
useUnifiedTopology: true });
```

```
const acsSchema = new mongoose.Schema({  
  name: String,  
  email: String  
});
```

```
const ACSModel = acsConnection.model('ACSModel', acsSchema);
```

```
// Example Schema and Model for the 'dcs' database
```

```
const dcsConnection = mongoose.createConnection(dcsUri, { useNewUrlParser: true,  
useUnifiedTopology: true });
```

```
const dcsSchema = new mongoose.Schema({  
  title: String,  
  description: String  
});
```

```
const DCSModel = dcsConnection.model('DCSModel', dcsSchema);
```

```
// Example usage - Create documents in both databases
```

```
// Creating a document in ACS database
```

```
const createACSData = async () => {  
  const newAcsData = new ACSModel({  
    name: 'John Doe',
```

```

    email: 'johndoe@acs.com'
  });

  await newAcData.save();
  console.log('Data saved in ACS');
};

// Creating a document in DCS database
const createDCSData = async () => {
  const newDcsData = new DCSModel({
    title: 'Introduction to DCS',
    description: 'A brief description of DCS'
  });

  await newDcsData.save();
  console.log('Data saved in DCS');
};

// Call functions to save data
createACSDData();
createDCSData();

```

Key Points:

1. **mongoose.createConnection:** This method is used to establish connections to multiple databases. You can create separate connections for each database and then use those connections to interact with the respective databases.
2. **Schema and Model for Each Database:** You define the schema and model for each database separately, so they can be used independently for performing CRUD operations.
3. **Connecting to Different Databases:** In the example above, we have two databases (acs and dcs) that are being connected to separately using Mongoose's createConnection method.

How it Works:

- You can now interact with both the acs and dcs databases independently in your application. For example, if you want to store user data in the acs database, you'd use the ACSModel schema, and if you want to store data in the dcs database, you'd use the DCSModel schema.

